

Marine Species Classification

Using Deep Learning on the FathomNet Underwater Imagery Dataset

CIE 555 — Neural Networks and Deep Learning | Spring 2026

Ahmed Mohamed — 202200294 — Assignment 1 — Report

Abstract

We present a complete deep-learning pipeline for classifying marine organisms captured in deep-sea underwater imagery from the FathomNet dataset. The pipeline covers (i) a custom four-block convolutional neural network (CNN) trained from scratch under two hyperparameter configurations, (ii) transfer learning from a pre-trained ResNet-50 backbone under three distinct fine-tuning strategies, and (iii) out-of-distribution (OOD) detection using the Maximum Softmax Probability (MSP) baseline. The dataset comprises 12 known species classes (1,991 labelled images split 70/15/15 for training/validation/test) and 3 held-out OOD classes (330 images). The best-performing model — ResNet-50 with partial fine-tuning — achieves **86.47 % test accuracy** and a **macro F1 of 0.8608**, representing a 23-point improvement over the custom CNN baseline. OOD detection at a tuned MSP threshold of 0.76 yields a recall of 0.89 with precision of 0.68.

1. Dataset & Preprocessing

1.1 FathomNet and Species Selection

FathomNet is an open-access database of geo-referenced underwater imagery annotated with localised bounding boxes and concept labels. From the full taxonomy we selected **15 classes**: 12 known (in-distribution) species covering a broad range of marine organism types — soft corals, echinoderms, cnidarians, cephalopods, and filter feeders — plus 3 OOD classes withheld entirely from training and validation to simulate novel species encountered at inference time.

Selection criteria: (a) ≥ 100 cropped images available on disk, (b) visually distinct from peer classes to ensure meaningful classification difficulty, and (c) good taxonomic spread to evaluate generalisation.

Class	Type	Split	Train	Val	Test
Actiniaria	Sea anemone	Known	97	21	21
Crinoidea	Feather star	Known	97	21	21
Holothuroidea	Sea cucumber	Known	97	21	21
Ophiuroidea	Brittle star	Known	97	21	21
Pennatulacea	Sea pen	Known	97	21	21
Porifera	Sea sponge	Known	97	21	21
Sebastolobus	Rockfish	Known	97	21	21

Nanomia	Siphonophore	Known	97	21	21
Paragorgia arborea	Bubblegum coral	Known	97	21	21
Dosidicus gigas	Humboldt squid	Known	97	21	21
Beroe abyssicola	Comb jelly	Known	97	21	21
Bathochordaeus stygius	Giant larvacean	Known	97	21	21
Aegina citrea	Jellyfish	OOD	—	—	110
Pleuroncodes planipes	Pelagic crab	OOD	—	—	110
Acanthogorgia	Prickly sea fan	OOD	—	—	110

Table 1. Dataset composition. OOD classes (orange) are completely absent from training and validation. Approximate counts shown; actual splits are stratified.

1.2 Data Splits

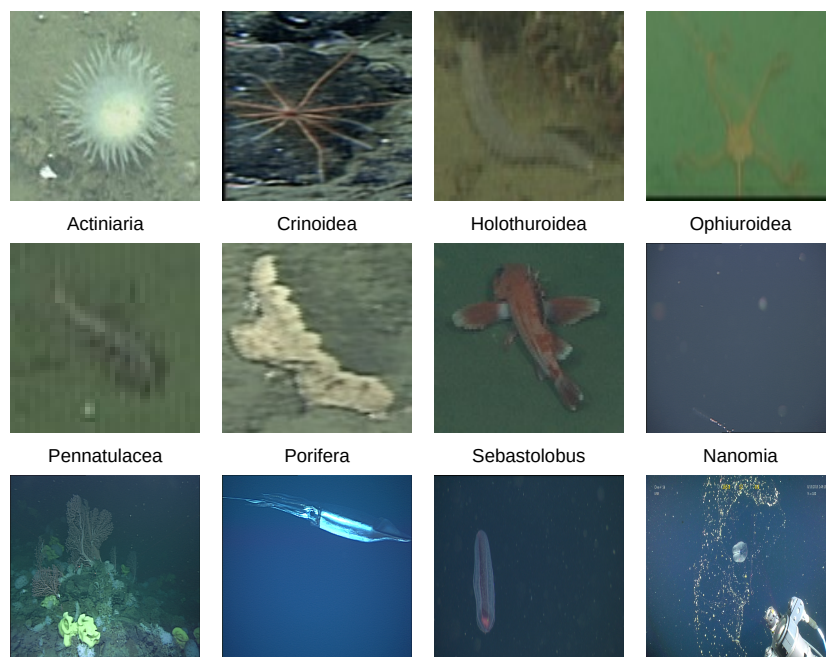
Known classes are split into **train / val / test = 70 / 15 / 15 %** using stratified sampling to ensure equal class representation. Split indices are persisted in *data/splits.json* for full reproducibility. OOD classes are never mixed with known-class splits; they form a dedicated OOD evaluation pool used only at test time. Final counts: **1,393 train · 295 val · 303 test · 330 OOD**.

1.3 Preprocessing & Augmentation

All images are resized to **224 × 224 px** (backbone compatibility). Training augmentation mimics the underwater environment: random horizontal flip, random resized crop (scale 0.7–1.0), and colour jitter (brightness ± 0.3 , contrast ± 0.3 , saturation ± 0.2 , hue ± 0.05) to compensate for depth-dependent colour casts and lighting variation. Evaluation images use a deterministic resize-to-256 → centre-crop-224 pipeline. All tensors are normalised to ImageNet mean/std ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$) for compatibility with pre-trained backbone weights.

1.4 Dataset Sample Images

Figure 1 shows one representative crop per class. Known classes (rows 1–2) are used for training; OOD classes (row 3) are withheld entirely and used only for OOD evaluation.



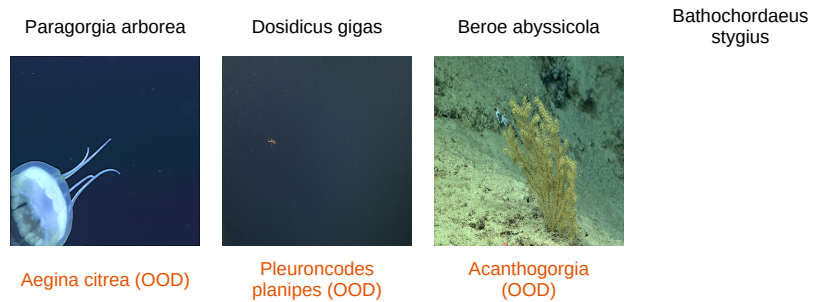


Figure 1. Representative crops from FathomNet. Rows 1–3: the 12 in-distribution training classes. Row 4 (orange labels): the 3 OOD classes withheld from all training.

2. Model A — Custom CNN from Scratch

2.1 Architecture

We designed a four-block convolutional network that progressively increases channel depth while halving spatial dimensions, followed by a fully-connected classification head. The full forward pass is:

Stage	Operation	Output shape
Input	—	$3 \times 224 \times 224$
Block 1	Conv(3→32, 3×3) → BN → ReLU → MaxPool	$32 \times 112 \times 112$
Block 2	Conv(32→64, 3×3) → BN → ReLU → MaxPool	$64 \times 56 \times 56$
Block 3	Conv(64→128, 3×3) → BN → ReLU → MaxPool	$128 \times 28 \times 28$
Block 4	Conv(128→256, 3×3) → BN → ReLU → AdaptiveAvgPool(4×4)	$256 \times 4 \times 4$
Flatten	—	4,096
FC 1	Linear(4096→512) → ReLU → Dropout(0.5)	512
FC 2	Linear(512→12)	12 (logits)

Table 2. MarineCNN layer-by-layer architecture.

Design rationale:

3 × 3 convolutions: Standard, parameter-efficient, and universally effective; two stacked 3×3 kernels cover the same receptive field as one 5×5 with fewer parameters. **Progressive channel doubling (32→64→128→256):** Increases representational capacity with depth while keeping early layers lightweight. **Batch normalisation before activation:** Stabilises gradient flow, reduces sensitivity to weight initialisation, and acts as a mild regulariser. **AdaptiveAvgPool(4×4) in the final block:** Decouples the FC head size from the spatial input resolution, making the model resolution-agnostic. **Dropout(0.5) on the FC head:** Primary regulariser; the overparameterised 512-unit head is most prone to overfitting on a ~1,400-sample dataset. Weights are initialised with Kaiming-uniform (He) initialisation.

2.2 Training Setup

Loss function: **CrossEntropyLoss with label smoothing $\epsilon = 0.1$** — smoothing prevents overconfident predictions and acts as a regulariser on a small dataset. Optimiser: **AdamW** (decoupled weight decay = $1e-4$) for reliable convergence with weight regularisation. Scheduler: **CosineAnnealingLR** over 30 epochs — smoothly anneals the learning rate to near-zero, avoiding abrupt step-decay artefacts. Training uses **Automatic Mixed Precision (AMP)** with gradient clipping (max_norm = 1.0) on an NVIDIA GeForce RTX 3060 (6.4 GB VRAM). Best-epoch checkpointing is performed on validation loss.

2.3 Hyperparameter Configurations

Two configurations are trained and compared to assess sensitivity to learning rate and batch size:

Parameter	Config 1 (Baseline)	Config 2
Learning rate	$1e-3$	$3e-4$
Batch size	32	64
Weight decay	$1e-4$	$1e-4$
Label smoothing	0.1	0.1

Epochs	30	30
Scheduler	CosineAnnealingLR	CosineAnnealingLR
Test Accuracy	63.04 %	61.39 %
Macro F1	0.6010	0.5913

Table 3. Model A hyperparameter configurations and final test metrics. Config 1 outperforms Config 2, suggesting the lower LR in Config 2 combined with a larger batch yields slower effective learning.

2.4 Training Curves

Model A — Config 1 (lr=1e-3, bs=32)

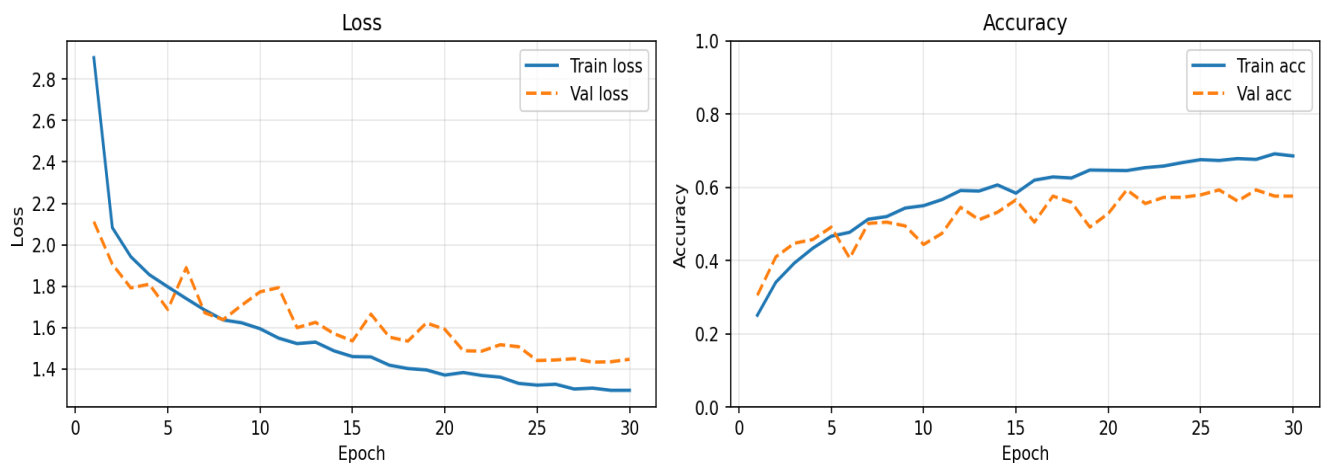


Figure 2. Model A Config 1 (lr=1e-3, batch=32) — loss and accuracy over 30 epochs.

Model A — Config 2 (lr=3e-4, bs=64)

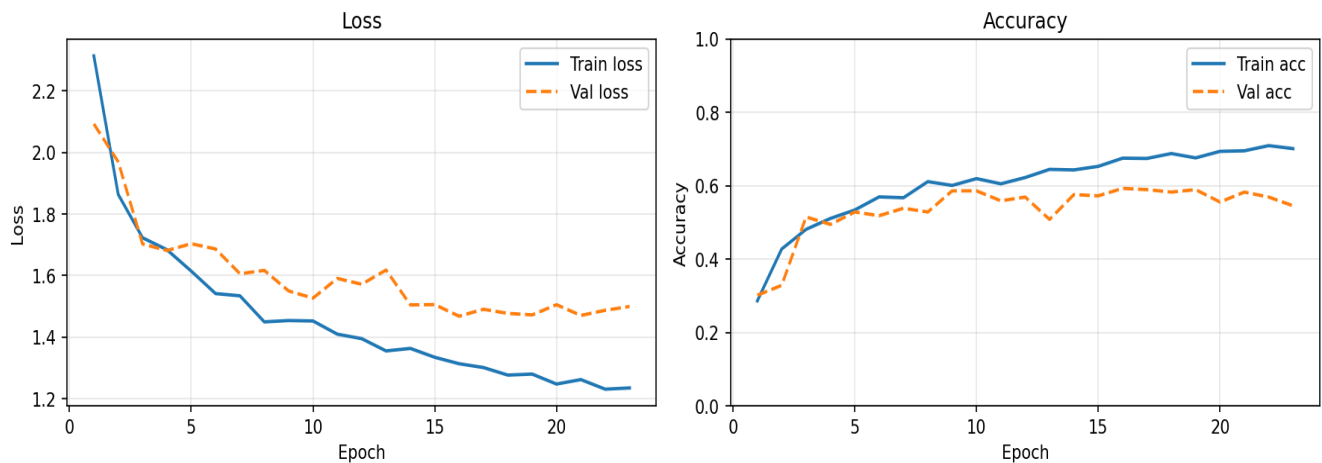


Figure 3. Model A Config 2 (lr=3e-4, batch=64) — loss and accuracy over 30 epochs.

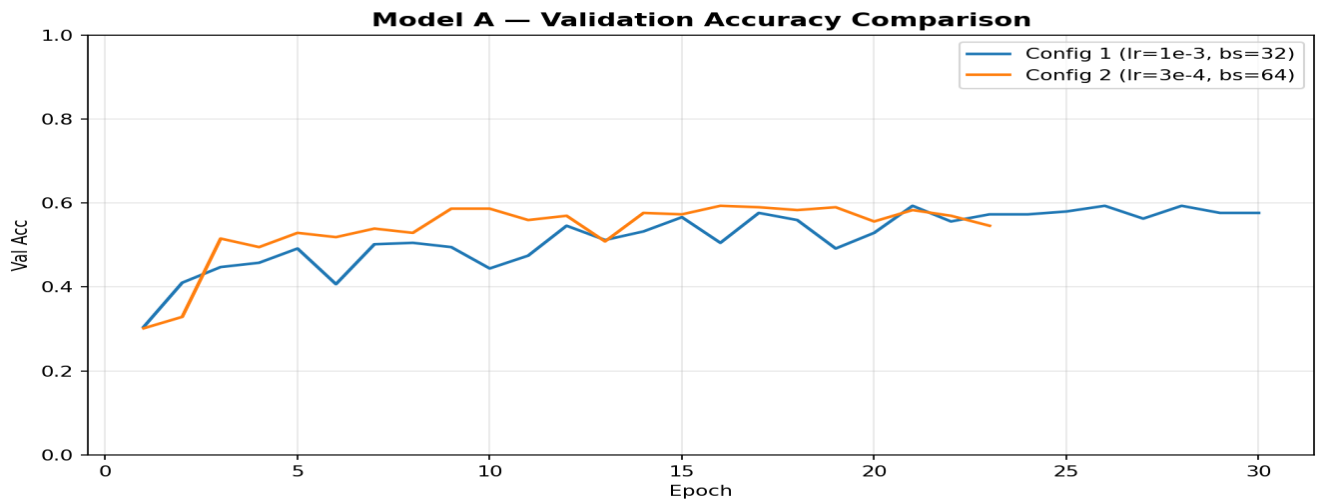


Figure 4. Side-by-side validation accuracy comparison between the two configurations.

2.5 Confusion Matrices

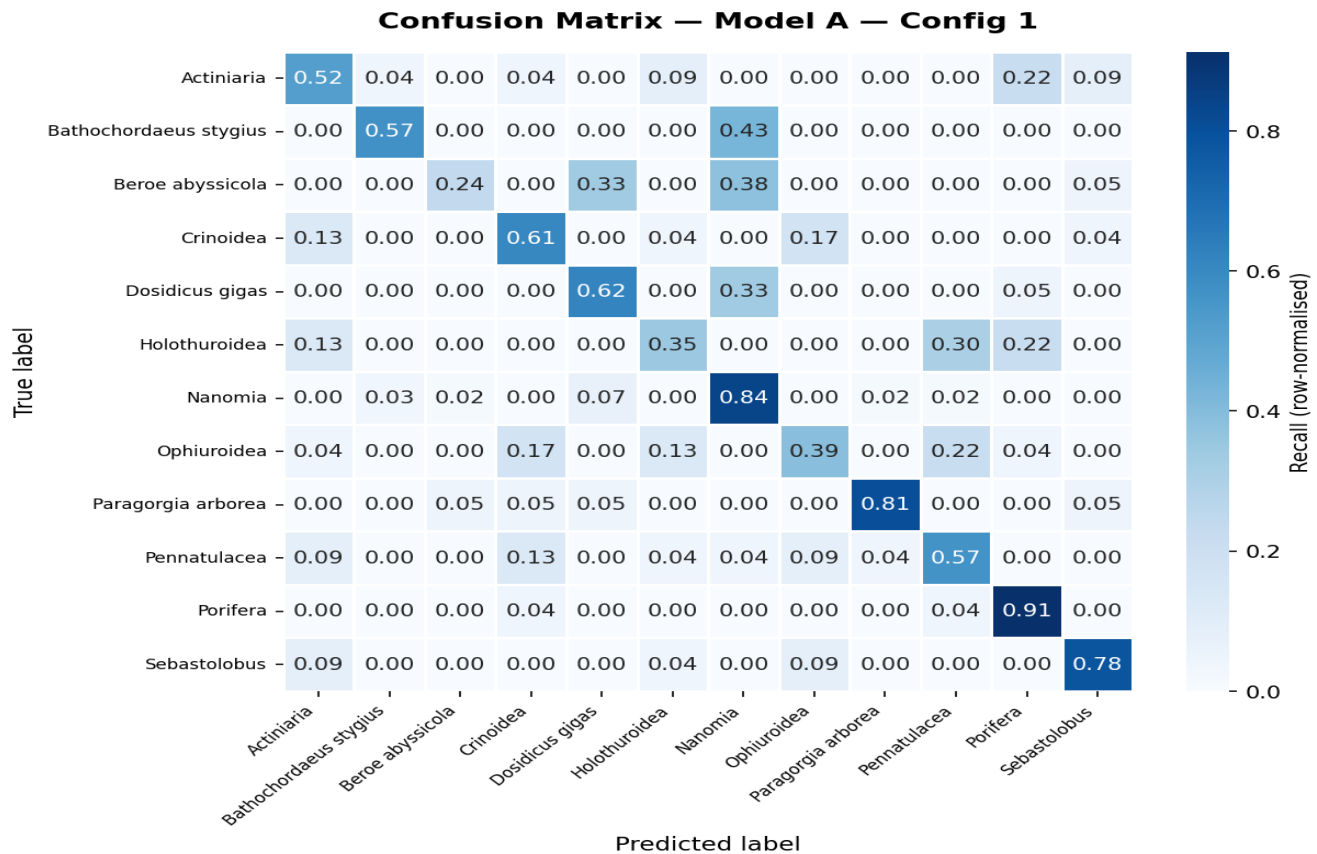


Figure 5. Normalised confusion matrix — Model A Config 1 on the test set.

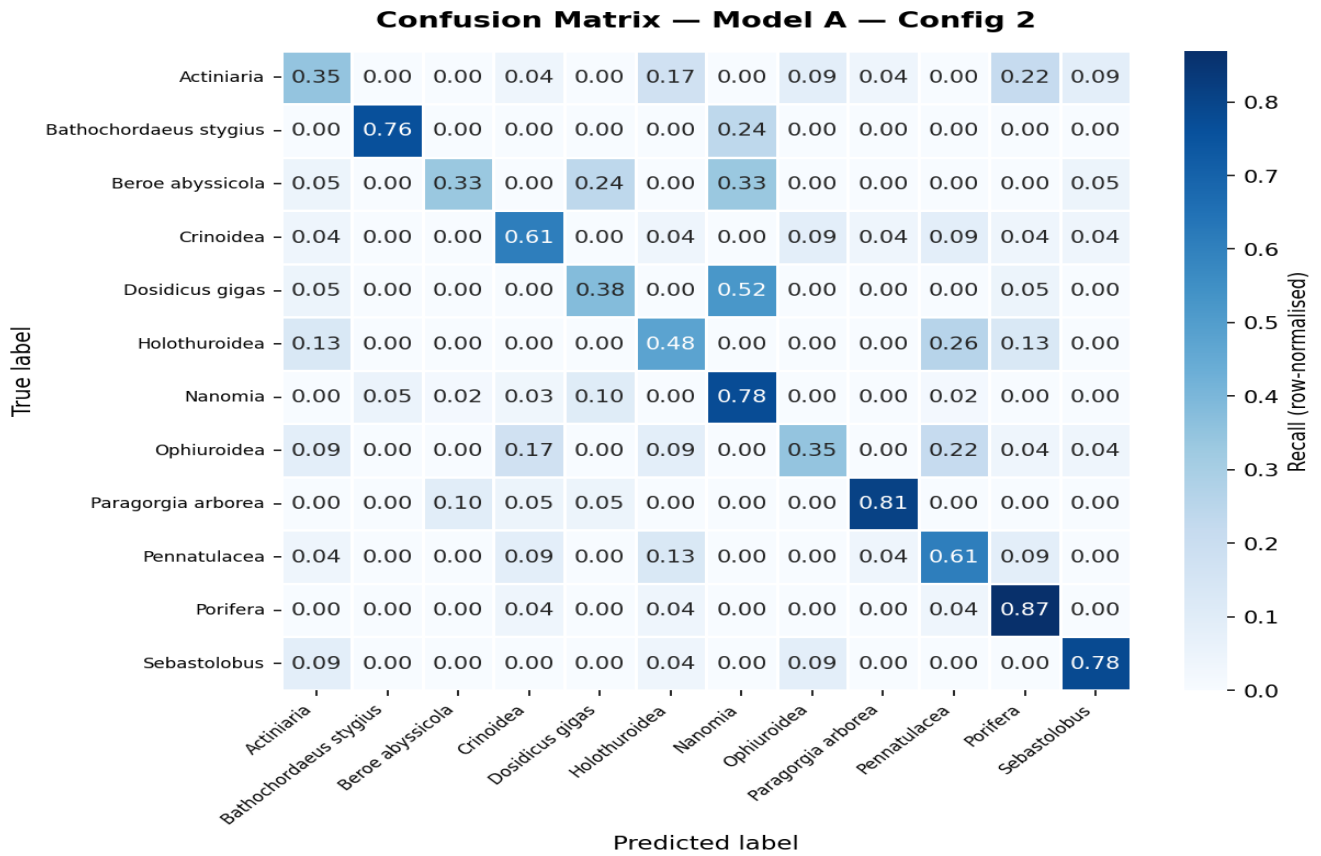


Figure 6. Normalised confusion matrix — Model A Config 2 on the test set.

2.6 Analysis

Config 1 achieves 63.04 % accuracy, slightly above Config 2 (61.39 %). Both configurations plateau after ≈ 20 epochs and show a widening gap between training and validation accuracy, indicating the onset of overfitting on the small dataset. The confusion matrices reveal that classes with similar textures (e.g. Porifera vs. Pennatulacea) are the primary source of confusion. Performance is inherently limited by the absence of pre-trained features; the network must learn visual representations from only $\sim 1,400$ samples.

3. Model B — Transfer Learning (ResNet-50)

3.1 Backbone and Head

We adopt **ResNet-50** pre-trained on ImageNet-1k as the feature backbone. The original 1,000-class classification head is replaced with a domain-adapted head designed to handle the smaller dataset and fewer target classes:

$$FC(2048 \rightarrow 512) \rightarrow ReLU \rightarrow Dropout(0.3) \rightarrow FC(512 \rightarrow 12)$$

Dropout(0.3) in the head provides regularisation while being lighter than the 0.5 used in Model A, as the pre-trained backbone already provides strong generalisation. All backbone layers use Kaiming initialisation inherited from pre-training.

3.2 Three Transfer Learning Strategies

Strategy A — Feature Extraction: All ResNet-50 backbone parameters are frozen. Only the 512-unit head (~1.06M parameters) is optimised. LR = 1e-3. This treats the backbone as a fixed feature extractor; faster per epoch but limited by the mismatch between ImageNet and underwater visual statistics.

Strategy B — Partial Fine-Tuning: Layers up to (and including) *layer3* are frozen; *layer4* (last residual stage) and the head are unfrozen (~16M parameters). Differential learning rates are used: backbone LR = 1e-4, head LR = 1e-3. This balances the risk of catastrophic forgetting against the benefit of adapting high-level features.

Strategy C — Full Fine-Tuning: The entire network (~24.6M parameters) is unfrozen. Very conservative backbone LR = 1e-5 to prevent destroying pre-trained features; head LR = 1e-4. A 3-epoch linear warmup precedes cosine decay annealing to allow the head to stabilise before updating the backbone.

Strategy	Trainable Params	LR (backbone)	LR (head)	Test Acc.	Macro F1
Feature Extraction	1,055,244	frozen	1e-3	79.54 %	0.7836
Partial Fine-Tuning	16,019,980	1e-4	1e-3	86.47 %	0.8608
Full Fine-Tuning	24,563,276	1e-5	1e-4	85.15 %	0.8452

Table 4. Transfer learning strategy comparison. Partial fine-tuning (green) achieves the best accuracy and macro F1.

3.3 Training Curves

Model B — Feature Extraction

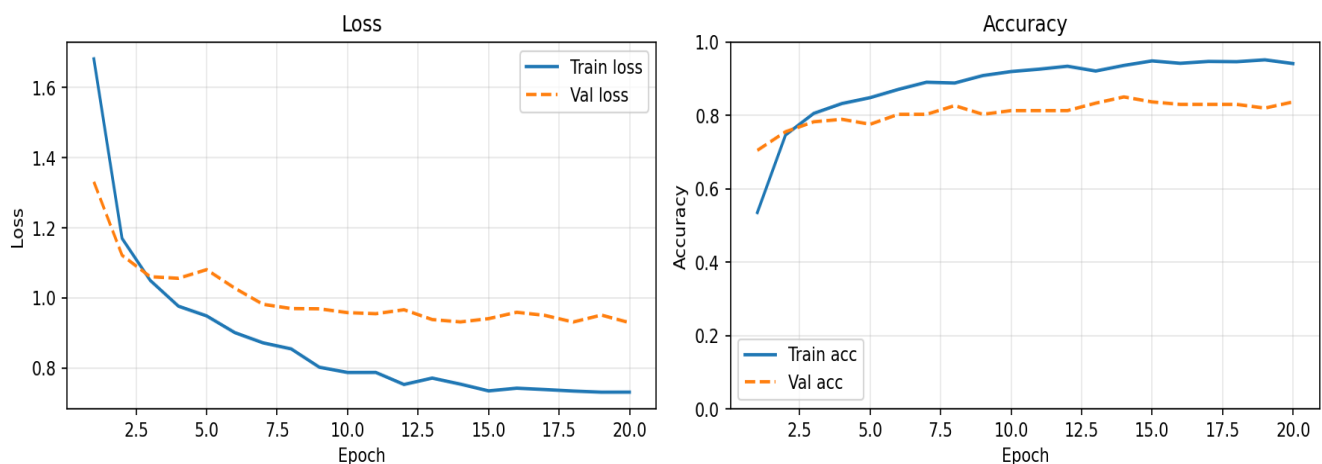


Figure 7. Feature extraction — training and validation curves.

Model B — Partial Fine-Tuning

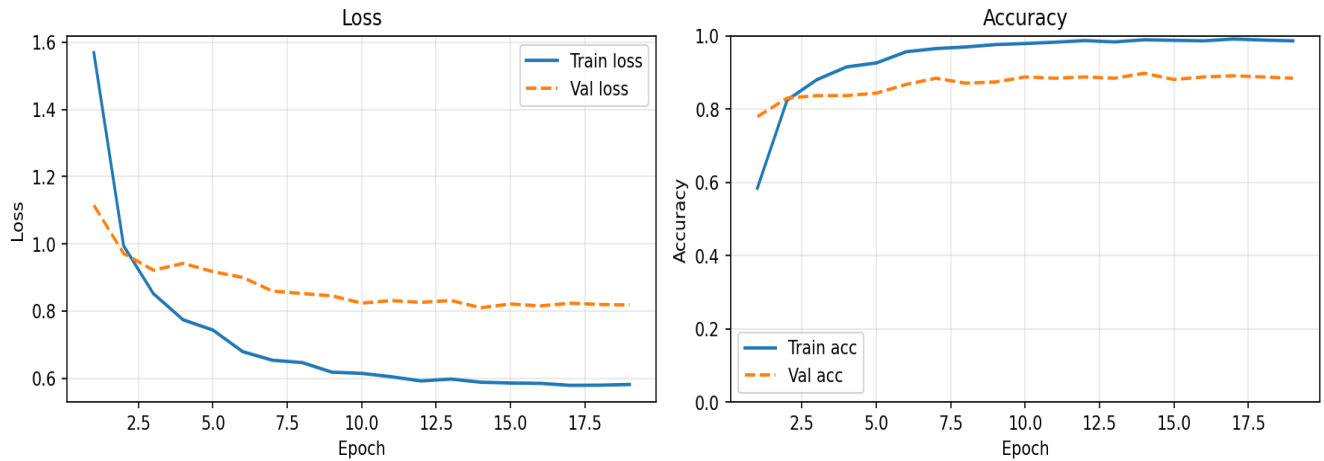


Figure 8. Partial fine-tuning — training and validation curves.

Model B — Full Fine-Tuning

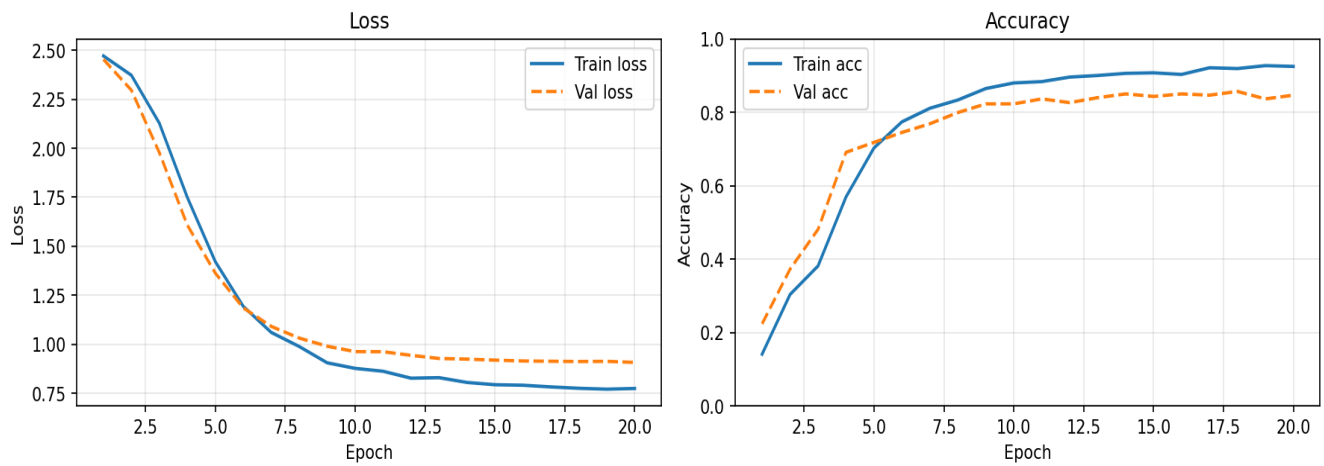


Figure 9. Full fine-tuning — training and validation curves.

Model B — Strategy Comparison (Val Accuracy)

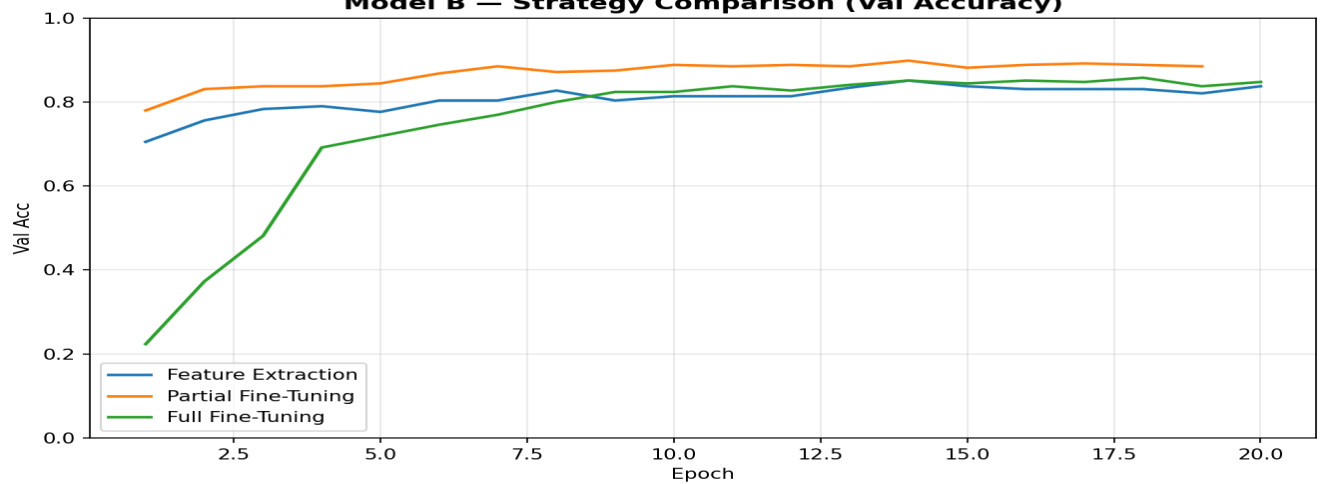


Figure 10. Validation accuracy across all three transfer learning strategies.

3.4 Confusion Matrices

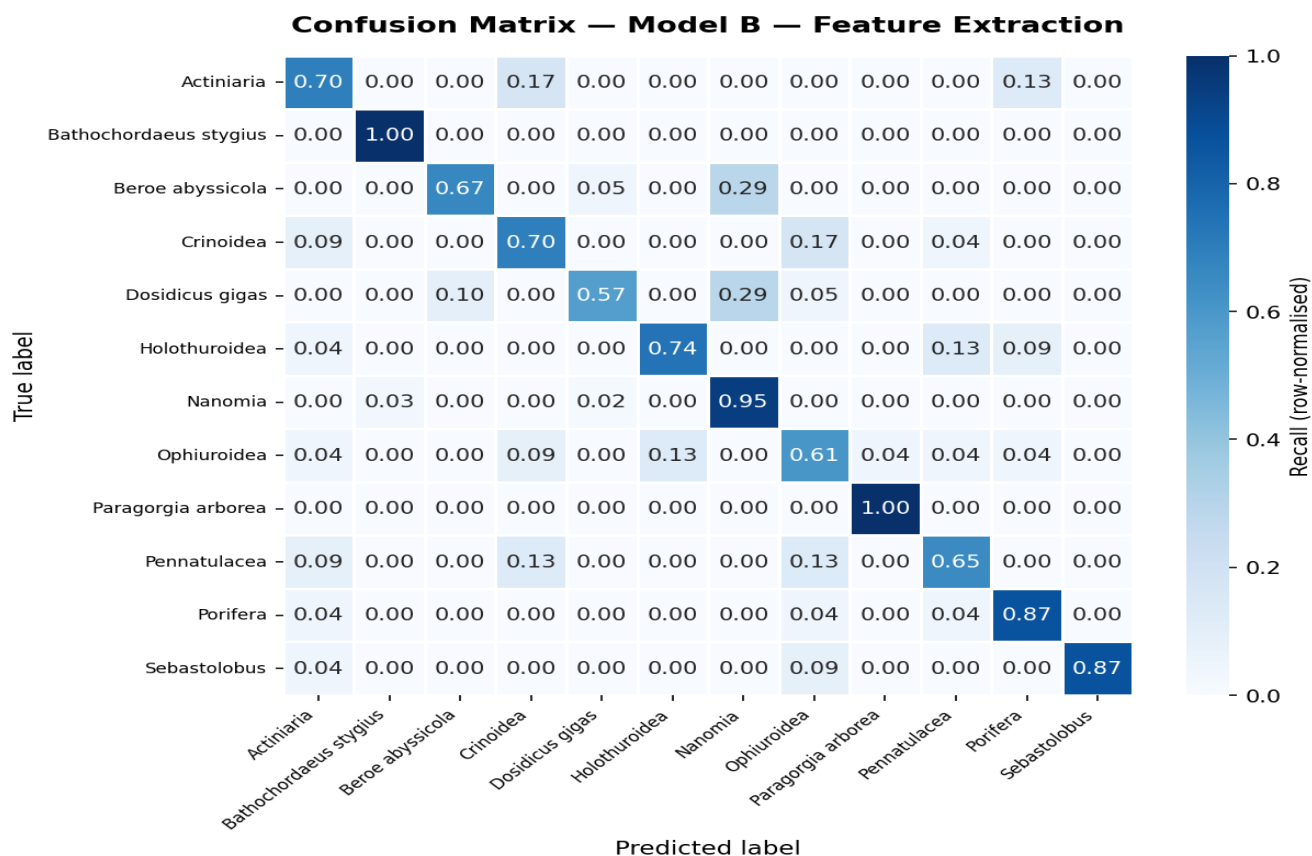


Figure 11. Normalised confusion matrix — Feature Extraction strategy.

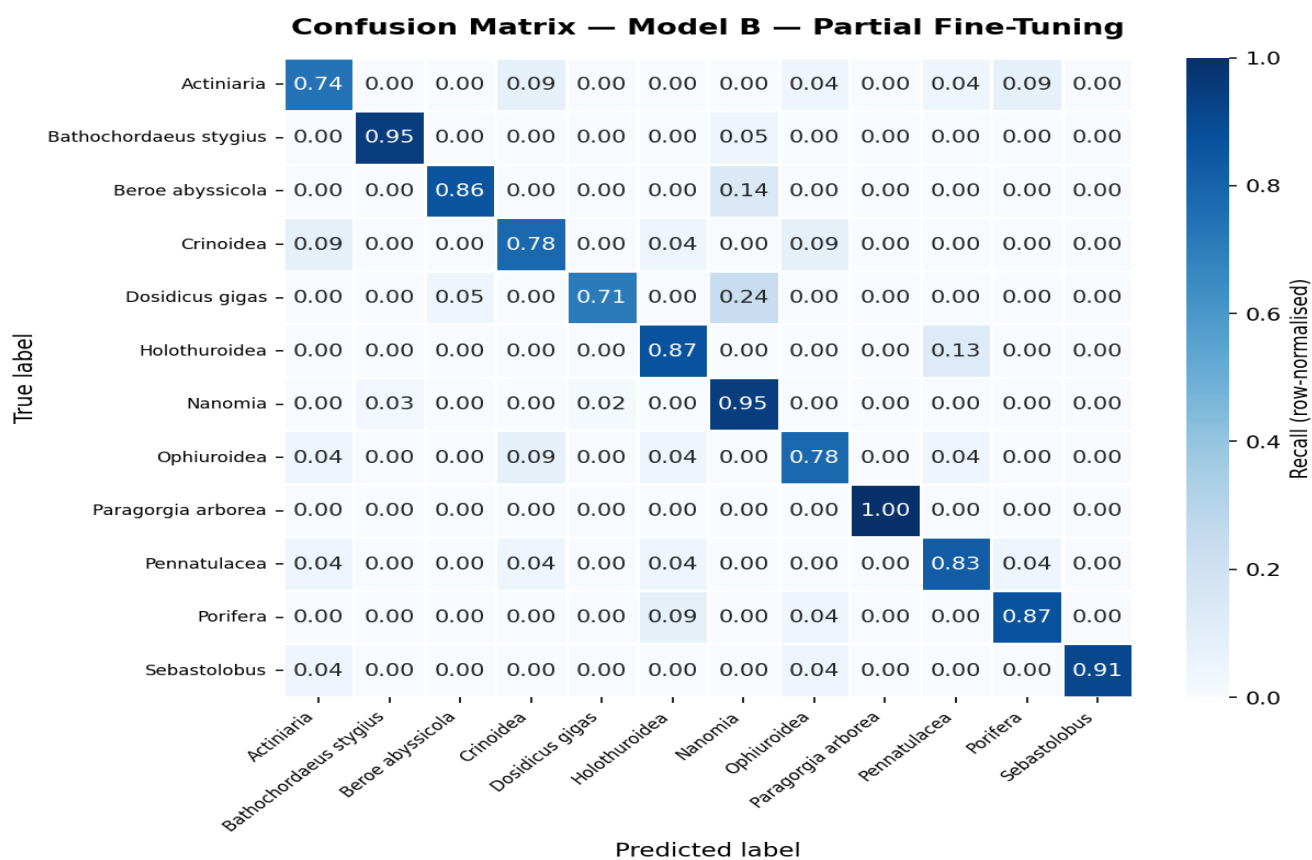


Figure 12. Normalised confusion matrix — Partial Fine-Tuning strategy (best model).

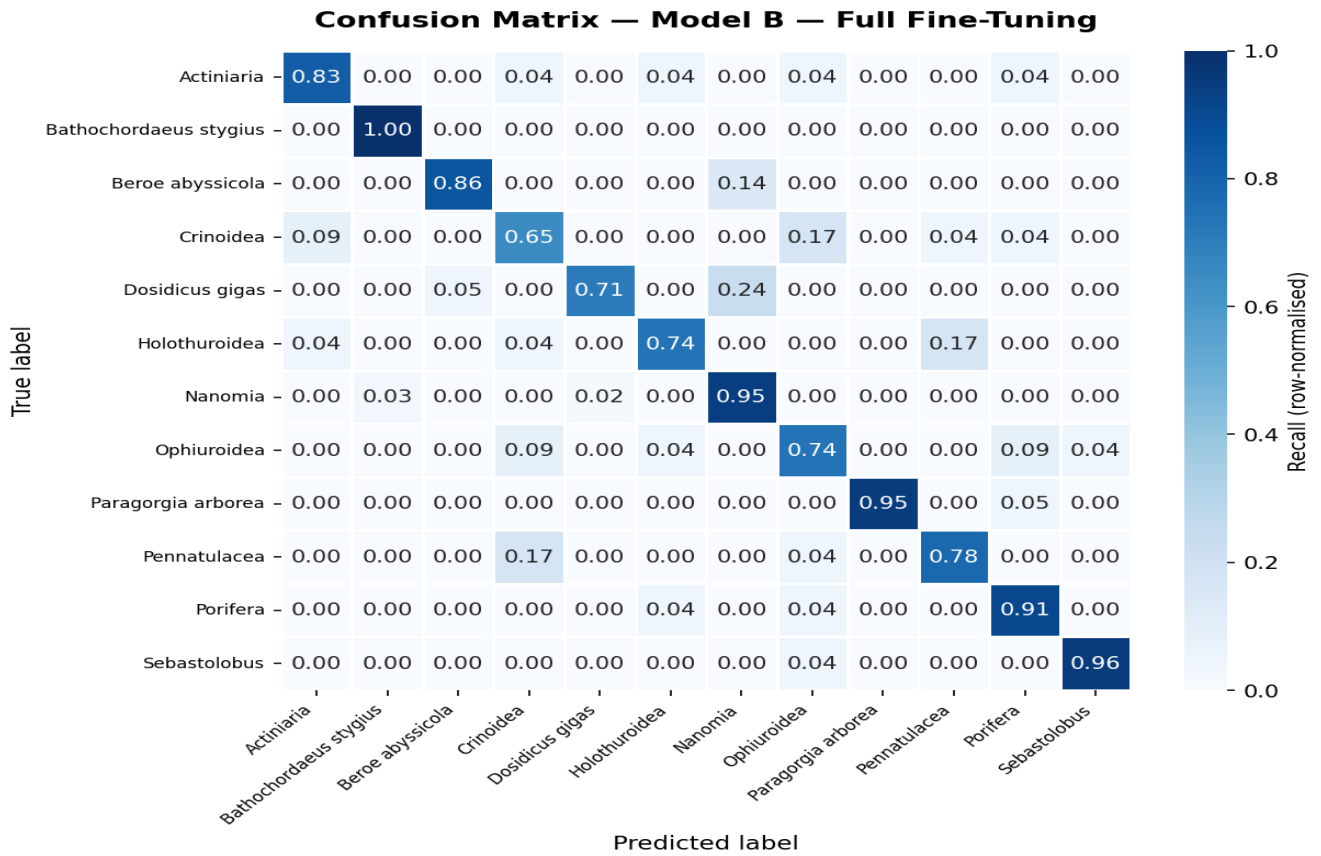


Figure 13. Normalised confusion matrix — Full Fine-Tuning strategy.

3.5 Analysis

Partial fine-tuning is the optimal strategy on this dataset. Feature extraction (Strategy A) reaches 79.54 % — a strong baseline that already outperforms the from-scratch CNN by over 16 points, confirming the value of pre-trained ImageNet representations even for underwater imagery. Unfreezing layer4 (Strategy B) provides a further 7-point gain because ResNet-50's layer4 encodes high-level object-part features that benefit from domain adaptation. Full fine-tuning (Strategy C) slightly underperforms partial fine-tuning (85.15 % vs. 86.47 %), likely because the very small dataset (1,393 training images) does not provide sufficient gradient signal to profitably update all 24.6M parameters without overfitting, even at $LR = 1e-5$.

4. Out-of-Distribution (OOD) Detection

4.1 Method — Maximum Softmax Probability (MSP)

We implement the MSP baseline (Hendrycks & Gimpel, 2017) as the OOD detector. For any input x the model's confidence is:

$$\text{confidence}(x) = \max_k \text{softmax}(f(x))_k$$

A sample is flagged as OOD if its confidence falls below a learned threshold τ . Intuitively, the network should be uncertain (low max-probability) about images from classes it has never seen. The threshold is tuned on the validation split by maximising the F1 score for OOD detection over a sweep of 200 candidate values in $[0, 1]$.

4.2 OOD Evaluation Results

OOD detection is performed using the best model (ResNet-50, partial fine-tuning). The in-distribution test set (303 images, 12 known classes) is combined with the OOD test pool (330 images, 3 unknown classes). A sample is positive (unknown) if its MSP is below the tuned threshold.

Metric	Value
Tuned threshold τ	0.7606
OOD Precision	0.6759
OOD Recall	0.8909
OOD F1 score	0.7686
False Positive Rate	0.4653

Table 5. OOD detection metrics at the optimal threshold.

OOD Detection — MSP Threshold (Model B Full Fine-Tuning)

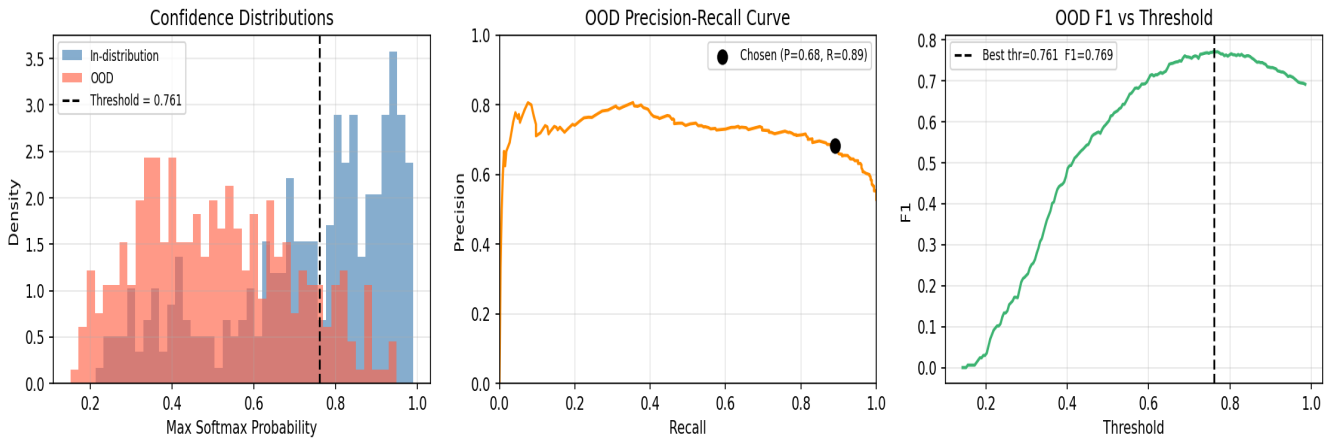


Figure 14. OOD detection results. Left: confidence score distributions for in-distribution (blue) and OOD (orange) samples with threshold marked. Centre: precision-recall curve over the threshold sweep. Right: F1 score as a function of threshold.

4.3 Analysis

At threshold $\tau = 0.76$, recall of 0.89 means the detector flags 89 % of OOD images as unknown — a strong result for a zero-shot method with no OOD-specific training. However, the false positive rate of 0.47 indicates that nearly half of known-class samples whose confidence falls just below τ are incorrectly flagged as unknown. This precision-recall trade-off is inherent to the MSP approach: the confidence distributions of in-distribution and OOD samples overlap significantly, as seen in the left panel of Figure 14. The F1-vs-threshold curve shows a broad plateau near $\tau \approx 0.75$ – 0.80 , giving reasonable robustness to threshold choice. More sophisticated OOD methods

(ODIN, energy-based, Mahalanobis distance) would likely improve separation at the cost of additional computation.

5. Consolidated Results

5.1 All-Model Comparison

Model	Trainable Params	Test Acc.	Macro F1	Rank
CNN (Config 1, lr=1e-3)	~0.55M	63.04 %	0.6010	5
CNN (Config 2, lr=3e-4)	~0.55M	61.39 %	0.5913	6 (last)
ResNet-50 Feat. Extract.	1.06M	79.54 %	0.7836	4
ResNet-50 Partial FT	16.02M	86.47 %	0.8608	1 (best)
ResNet-50 Full FT	24.56M	85.15 %	0.8452	2

Table 6. Full model comparison. Transfer learning consistently outperforms the from-scratch CNN. Partial fine-tuning achieves the best overall performance.

5.2 Key Observations

Transfer learning gap: Even feature extraction alone (no backbone updates) surpasses the best from-scratch model by 16.5 percentage points (79.54 % vs. 63.04 %). This is particularly striking given that FathomNet underwater imagery differs substantially from ImageNet. Mid-level features learned from ImageNet (edges, textures, shapes) transfer effectively across domains.

Diminishing returns from full fine-tuning: Unfreezing the entire backbone (Strategy C) does not improve over partial fine-tuning (Strategy B) despite tripling the trainable parameter count. The additional 8.5M parameters in the earlier residual blocks do not learn useful task-specific features from only 1,393 training images.

CNN regularisation: The from-scratch CNN achieves a reasonable 63 % given dataset size, validating the Dropout + BatchNorm + label smoothing regularisation strategy. Additional data augmentation or a pre-training step on a larger unlabelled underwater dataset would narrow the transfer learning gap.

OOD detection: The MSP baseline provides practical OOD detection with high recall (0.89) at the cost of moderate precision (0.68). This trade-off is appropriate for a marine monitoring system where missing novel species (false negatives) is more costly than flagging known species for expert review (false positives).

6. Discussion and Limitations

6.1 Dataset Limitations

The 15-class FathomNet subset used here (~2,300 total images) is small by modern deep learning standards. Class sizes are approximately balanced (~140 images/class for known classes) but individual images vary widely in quality, resolution, and background complexity due to the diversity of camera systems used to collect FathomNet data. Future work should expand to more classes and incorporate multi-label annotations (multiple organisms per image).

6.2 Model Limitations

The CNN architecture is intentionally modest to remain trainable from scratch on limited data. Deeper architectures (ResNet-style skip connections) implemented from scratch would likely improve performance but require more careful regularisation. For transfer learning, only ResNet-50 was evaluated; EfficientNet-B0 is implemented in `src/model_b.py` and available as a drop-in swap.

6.3 OOD Method Limitations

MSP is a simple yet competitive OOD baseline but does not use any OOD-specific training signal. Methods such as ODIN (temperature scaling + input perturbation), energy-based OOD scoring, or Mahalanobis distance in the penultimate feature space would likely improve precision without sacrificing recall.

6.4 Reproducibility

All random seeds are fixed to 42 via *src/seed.py* covering Python built-in random, NumPy, PyTorch CPU and CUDA, and cuDNN deterministic mode. Dataset split indices are saved to *data/splits.json*. All dependency versions are pinned in *requirements.txt*. The notebook can be re-executed top-to-bottom on any machine with the same data folder to reproduce all results.

References

- Hendrycks, D., & Gimpel, K. (2017). A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks. *ICLR 2017*.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *CVPR 2016*, pp. 770–778.
- Ioffe, S., & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *ICML 2015*.
- Loshchilov, I., & Hutter, F. (2019). Decoupled Weight Decay Regularization. *ICLR 2019*.
- FathomNet. (2024). FathomNet: A Global Image Database for Enabling AI in the Ocean. <https://fathomnet.org>.
- Torchvision contributors. (2024). torchvision 0.24. <https://github.com/pytorch/vision>.